Coin Change I (Minimum Coins)

Complete Recursion + DP Documentation

Problem Statement

Given an array of coin denominations and a target amount, find the minimum number of coins required to make the amount. You may use any coin unlimited times.

Example:

`coins = [1,3,4]`

`amount = 6`

Answer = 2

Because: `3 + 3 = 6` uses only 2 coins.

Solution Code

```
int coinChange(vector<int>& coins, int amount) {
    vector<int> dp(amount + 1, -2);
    return helper(coins, amount, dp);
}

int helper(vector<int>& coins, int rem, vector<int>& dp) {
    if(rem == 0)
        return 0;

    if(rem < 0)
        return -1;

    if(dp[rem] != -2)
        return dp[rem];

    int mini = INT_MAX;

    for(int coin : coins) {
        int res = helper(coins, rem - coin, dp);

        if(res >= 0 && res < mini)
            mini = 1 + res;
    }

    dp[rem] = (mini == INT_MAX) ? -1 : mini;
    return dp[rem];
}
```

Core State Definition

`helper(rem)` means: Minimum number of coins required to make the remaining amount `rem`.

> **Example:**
> `helper(6)` asks: What is the minimum number of coins needed to make amount 6?

Why Is There No Index?

In many DP problems, `solve(idx, target)` is used. Examples include:

- 0/1 Knapsack
- Subset Sum
- Coin Change II (Count Ways)

because decisions depend on which items are available. In Coin Change I, `helper(rem)` is sufficient.

Why?

Because at every step: All coins are always available. If `coins = [1,3,4]` and amount is `6`, we can choose `1`, `3`, or `4` at every recursive call. Therefore, only the remaining amount matters.

Recursive Thinking

Suppose: `helper(6)`. We try every coin.

- Coin = 1 → `1 + helper(5)`
- Coin = 3 → `1 + helper(3)`
- Coin = 4 → `1 + helper(2)`

Therefore:

$$helper(6) = min(1 + helper(5), 1 + helper(3), 1 + helper(2))$$

This is the main recurrence.

Base Case 1

```
if(rem == 0)
    return 0;
```

Meaning: Amount has been completely formed. No more coins are needed.

> **Example:** `helper(0)` returns `0` because we already reached the target.

Base Case 2

```
if(rem < 0)
    return -1;
```

Meaning: Invalid path.

> **Example:** `amount = 3` , `coin = 5` . Remaining amount: `helper(-2)` . Impossible. Return: `-1`

Understanding mini

Inside every recursive call: `int mini = INT_MAX;` . This variable stores the best (minimum) answer found for the current amount.

Important: Each recursive call gets its own `mini` .

> **Example:**
> `helper(6)` creates: `mini = INT_MAX`
> Then `helper(5)` creates another: `mini = INT_MAX`
> Then `helper(4)` creates another: `mini = INT_MAX`

These variables are completely independent. Each recursive stack frame has its own copy.

What Happens Inside One Call?

Suppose `helper(6)` starts. Initially: `mini = INT_MAX` . Loop: `for(int coin : coins)` tries all possible first coins.

Understanding the Return Value

This is the most important concept. When we do: `res = helper(rem - coin);` we are asking: *What is the minimum number of coins needed for the remaining amount?* **NOT:** *How many coins did this specific path use?*

The recursive call always returns the optimal answer for the smaller problem.

Example

Coins: `[1,3,4]` , Amount: `6`

Branch 1

Take coin: `1` . Then: `helper(5)` . Suppose: `helper(5) = 2` . Meaning: The minimum coins needed for amount 5 is 2. Therefore: `1 + helper(5) = 1 + 2 = 3` . This branch requires: **3 coins**.

Branch 2

Take coin: `3`. Then: `helper(3)`. Suppose: `helper(3) = 1` because 3 can be formed using one coin (3). Therefore: `1 + helper(3) = 1 + 1 = 2`. This branch requires: **2 coins**. Path: `3 + 3`

Branch 3

Take coin: `4`. Then: `helper(2)`. Suppose: `helper(2) = 2` because 1 + 1. Therefore: `1 + helper(2) = 1 + 2 = 3`. This branch requires: **3 coins**. Path: `4 + 1 + 1`

Choosing the Best Answer

Now we have: Branch 1 = 3, Branch 2 = 2, Branch 3 = 3. We choose: `mini = min(3,2,3)`. Result: `mini = 2`. Therefore: `helper(6) = 2`

Common Misconception

Many students think the returns from different branches are added together. That is incorrect. We are **NOT** doing:

```
return branch1 + branch2 + branch3;
```

Instead:

```
candidate1 = 1 + helper(5);
candidate2 = 1 + helper(3);
candidate3 = 1 + helper(2);
return min(candidate1, candidate2, candidate3);
```

The branches compete against each other. Only the best one survives.

Why Do We Add 1 + res?

Suppose `helper(3)` returns `1`. Meaning: Amount 3 needs 1 coin. If we first chose coin `3` for amount `6`, then:

- Current coin = 1 coin
- Remaining amount = 3
- Best solution for remaining = 1 coin
- Total: `1 + 1 = 2`

Hence, `1 + res` means: Current coin + Optimal solution of the remaining amount.

DP Meaning

`dp[x]` stores: Minimum number of coins required to make amount x.

> **Example:** `dp[6] = 2` means amount 6 can be formed using minimum 2 coins.

Why Memoization Works

Without DP, `helper(3)` may be computed many times. Example:

- `helper(6)` → `helper(5)` → `helper(3)`
- `helper(6)` → `helper(3)`

Same state repeats. With memoization:

```
if(dp[rem] != -2)
    return dp[rem];
```

Once `dp[3] = 1` is stored, future calls instantly return `1` without recomputation.

Complete Algorithm

For every amount:

Step 1

Try every coin: `for(coin : coins)`

Step 2

Reduce the amount: `helper(rem - coin)`

Step 3

Ask recursion: What is the optimal answer for the remaining amount?

Step 4

Add current coin: `1 + res`

Step 5

Keep the minimum answer: `mini = min(mini, 1 + res)`

Step 6

Store answer: `dp[rem] = mini`

Step 7

Return answer: `return dp[rem]`

Mathematical Recurrence

$$helper(rem) = min(1 + helper(rem - coin_1), 1 + helper(rem - coin_2), 1 + helper(rem - coin_3), ...)$$

(Ignoring invalid states)

Pattern Identification

This is **NOT** a Take/Not-Take DP. Examples of Take/Not-Take DP include 0/1 Knapsack, Subset Sum, and Coin Change II, where the state is `(idx, target)`.

Coin Change I belongs to **Choice Enumeration DP** or **State Reduction DP**.

General Pattern

$$f(state) = best\ among\ all\ possible\ next\ moves$$

Examples using the same pattern:

- **Coin Change I:** `amount → amount - coin`
- **Perfect Squares:** `n → n - square`
- **Minimum Steps To One:** `n → n/2`, `n → n/3`, `n → n-1`
- **Word Break:** `string → remaining suffix`

Complete Dry Run (How Values Return and How Minimum Is Chosen)

Let's do a full dry run for: `coins = [1,3,4]`, `amount = 6`. We start with `helper(6)`.

Step 1: Evaluate helper(6)

$$helper(6) = min(1 + helper(5), 1 + helper(3), 1 + helper(2))$$

To compute this, recursion first needs the values of: `helper(5)`, `helper(3)`, and `helper(2)`.

Step 2: Evaluate helper(2)

$$helper(2) = min(1 + helper(1), 1 + helper(-1), 1 + helper(-2))$$

Invalid states: `helper(-1) = -1`, `helper(-2) = -1`. Now compute `helper(1)`.

Evaluate helper(1):

$$helper(1) = min(1 + helper(0), 1 + helper(-2), 1 + helper(-3))$$

Base case: `helper(0) = 0`. Invalid: `helper(-2) = -1`, `helper(-3) = -1`. So `helper(1) = 1 + 0 = 1`. Store: `dp[1] = 1`. Return: `1`.

Back to `helper(2)`: Now Coin 1 → `1 + helper(1) = 1 + 1 = 2`. Coin 3 → invalid, Coin 4 → invalid. Minimum: `mini = 2`. Store: `dp[2] = 2`. Return: `2`.

Step 3: Evaluate helper(3)

$$helper(3) = min(1 + helper(2), 1 + helper(0), 1 + helper(-1))$$

We already know: `helper(2) = 2`, `helper(0) = 0`, `helper(-1) = -1`.

Candidates:

- Coin 1 → `1 + 2 = 3`
- Coin 3 → `1 + 0 = 1`
- Coin 4 → invalid

Minimum: `mini = 1`. Store: `dp[3] = 1`. Return: `1`.

Notice something important: `helper(3)` returns 1 because the best way to make amount 3 is `3` using only one coin.

Step 4: Evaluate helper(5)

$$helper(5) = min(1 + helper(4), 1 + helper(2), 1 + helper(1))$$

Need `helper(4)`. Let's evaluate `helper(4)`:

$$helper(4) = min(1 + helper(3), 1 + helper(1), 1 + helper(0))$$

Known values: `helper(3) = 1`, `helper(1) = 1`, `helper(0) = 0`.

Candidates: Coin 1 → `1 + 1 = 2`, Coin 3 → `1 + 1 = 2`, Coin 4 → `1 + 0 = 1`. Minimum: `mini = 1`. Store: `dp[4] = 1`. Return: `1`.

Back to `helper(5)`: Known values: `helper(4) = 1`, `helper(2) = 2`, `helper(1) = 1`. Candidates: Coin 1 → `1 + 1 = 2`, Coin 3 → `1 + 2 = 3`, Coin 4 → `1 + 1 = 2`. Minimum: `mini = 2`. Store: `dp[5] = 2`. Return: `2`.

Step 5: Finish helper(6)

Now all required values are known: `helper(5) = 2`, `helper(3) = 1`, `helper(2) = 2`. So:

$$helper(6) = min(1 + 2, 1 + 1, 1 + 2)$$

Candidates: Coin 1 → 3, Coin 3 → 2, Coin 4 → 3. Minimum: `mini = 2`. Store: `dp[6] = 2`. Return: `2`.

Final Answer

`helper(6) = 2` which corresponds to `6 = 3 + 3` using **2 coins**.

Key Observation From The Dry Run

Notice how each recursive call returns the best answer for its own amount:

- `helper(1)` returns 1
- `helper(2)` returns 2

- `helper(3)` returns 1
- `helper(4)` returns 1
- `helper(5)` returns 2
- `helper(6)` returns 2

Then the parent call does `1 + returned_value` because it must count the current coin as well. Finally, `mini = min(all candidates)` selects the best among all possible first coin choices.

This is the complete intuition behind Coin Change I (Minimum Coins) using Recursion + Memoization.